

智算集群AI业务高效与精细化的在线Profiling

定义AI业务观测新范式

霍智鑫/黄永琢

阿里云智能集团 弹性计算/基础软件

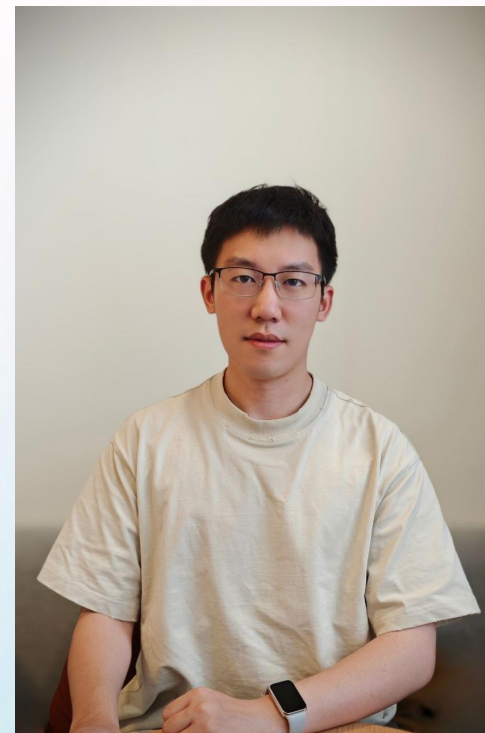
2025/11/15





霍智鑫

阿里云智能-弹性计算-高级研发工程师



黄永琢

阿里云智能-基础软件-高级研发工程师

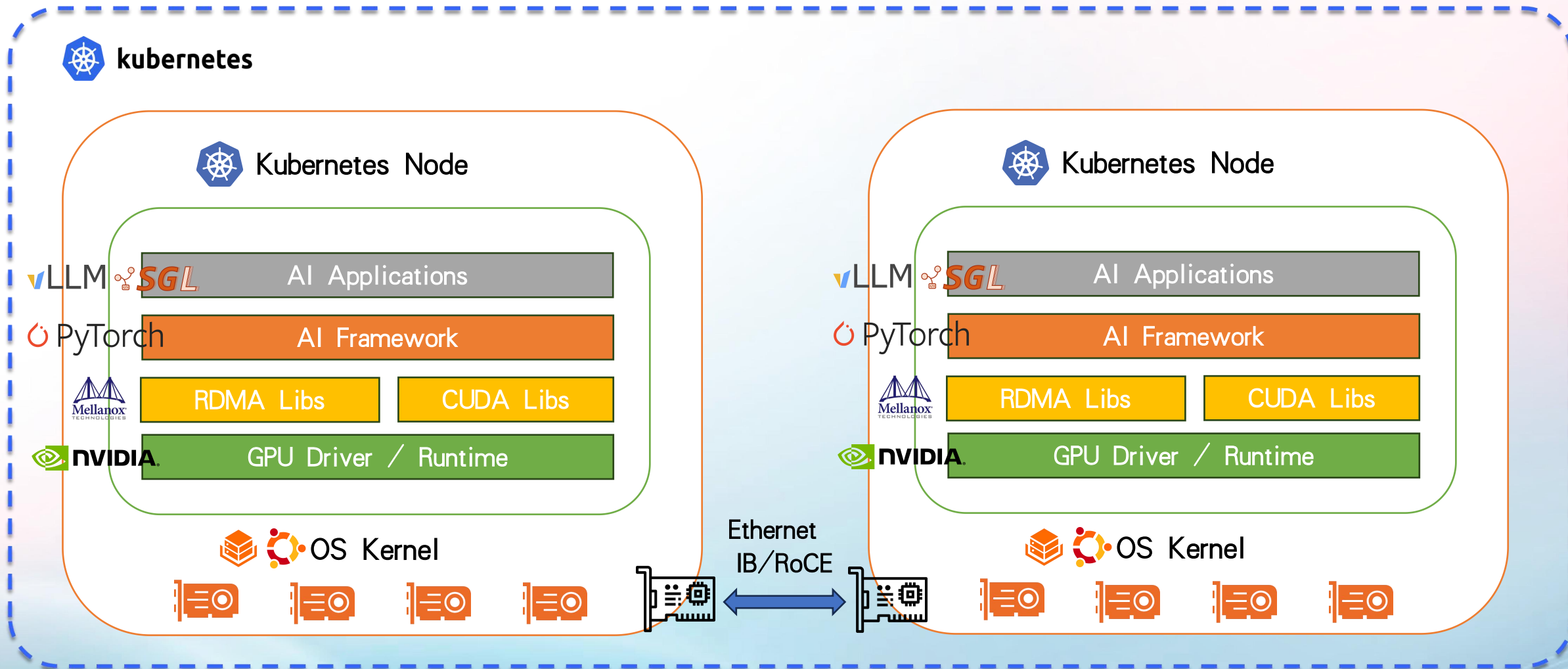
CONTENT

目录

- 01 在线AI业务挑战与Profiling示例
- 02 Kubernetes原生的AI Profiling架构
- 03 AI Profiling关键技术与效果验证
- 04 案例分析

01 在线AI业务挑战与Profiling示例

复杂的分布式AI业务（训练/推理）架构



引子：vLLM线上推理服务的显存泄漏问题

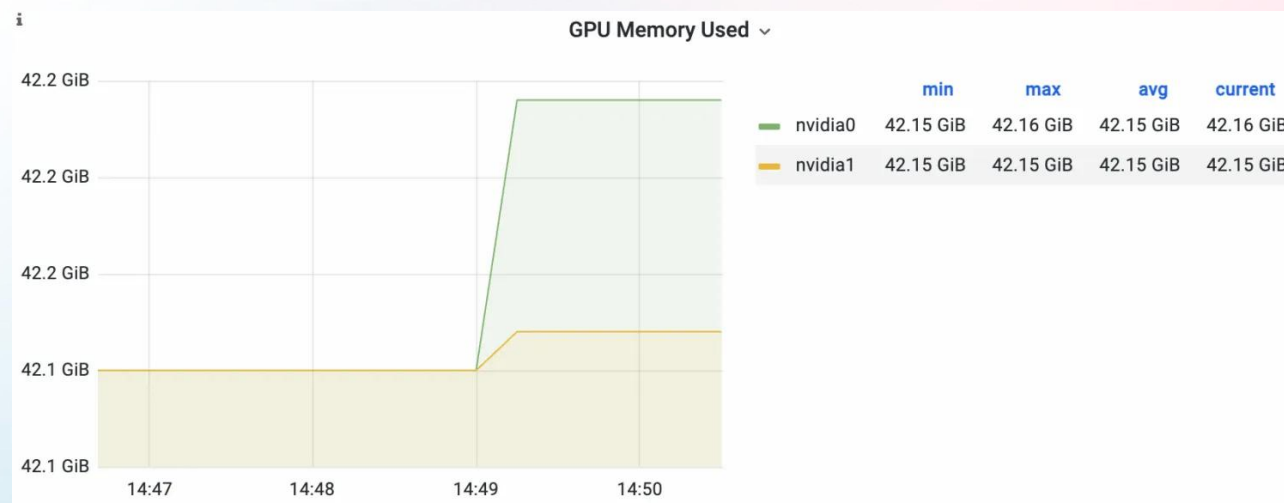
vLLM启动时使用显存预分配机制，在KV-Cache利用率未达满情况下理论显存值不应上涨

复现现象1：通过 nvidia-smi 观察到有显存增长

NVIDIA-SMI 550.144.03 Driver Version: 550.144.03 CUDA Version: 12.4									
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp		Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M.		
						MIG	M.		
0	NVIDIA L20		On	00000000:00:03.0	Off		0		
N/A	38C	P0	95W / 350W	43161MiB / 46068MiB	0%	Default			
									N/A
1	NVIDIA L20		On	00000000:00:04.0	Off		0		
N/A	39C	P0	97W / 350W	43161MiB / 46068MiB	0%	Default			
									N/A

NVIDIA-SMI 550.144.03 Driver Version: 550.144.03 CUDA Version: 12.4									
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp		Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M.		
						MIG	M.		
0	NVIDIA L20		On	00000000:00:03.0	Off		0		
N/A	37C	P0	93W / 350W	43187MiB / 46068MiB	0%	Default			
									N/A
1	NVIDIA L20		On	00000000:00:04.0	Off		0		
N/A	38C	P0	96W / 350W	43175MiB / 46068MiB	0%	Default			
									N/A

复现现象2：通过 dcgm-exporter 监控观察到有显存增长



复现现象3：观测vLLM日志发现KV-Cache利用率并未打满

```
INFO 04-25 03:44:52 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 39.4 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 0.3%, CPU KV cache usage: 0.0%.
INFO 04-25 03:44:57 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 39.2 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 0.6%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:02 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 39.1 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 0.8%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:07 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 38.9 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 1.1%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:12 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 38.8 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 1.3%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:17 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 38.7 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 1.5%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:22 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 38.7 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 1.8%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:27 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 38.7 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 2.0%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:32 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 38.7 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 2.3%, CPU KV cache usage: 0.0%.
INFO 04-25 03:45:37 metrics.py:341] Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 38.7 tokens/s, Running: 2 reqs, Swapped: 0 reqs, Pending: 0 reqs, GPU KV cache usage: 2.5%, CPU KV cache usage: 0.0%.
```

开盒业务Pod所可能用到的排查工具



➤ NVIDIA DCGM

- 优势：Kubernetes Pod与GPU维度监控数据，Metrics持续上报；
- 劣势：业务实际运行情况完全黑盒；



➤ NVIDIA Nsight

- 优势：能细粒度抓取GPU上CUDA Stream、CUDA Kernel、相关CPU占用等执行情况，以TimeLine形式呈现出来；
- 劣势：Overhead大，只能离线控制任务，无具体AI业务的执行栈 (Python)；



➤ bpftrace

- 优势：使用eBPF从内核层面提供CPU、Memory、Syscall、Uprobe等方面的trace数据；
- 劣势：输出内容与其他分析数据格式差异，结合分析门槛较高，且无GPU相关数据；



➤ Torch Profiler

- 优势：能抓取较为丰富的Torch层、CUDA Kernel层的数据，且也能以Trace格式输出；
- 劣势：需要显式在业务Python代码中声明，对业务代码具有侵入性，线上业务成本高。

02 Kubernetes原生的AI Profiling架构



Kubernetes原生的精细化AI Profiling能力

业务无侵入、代码零改造

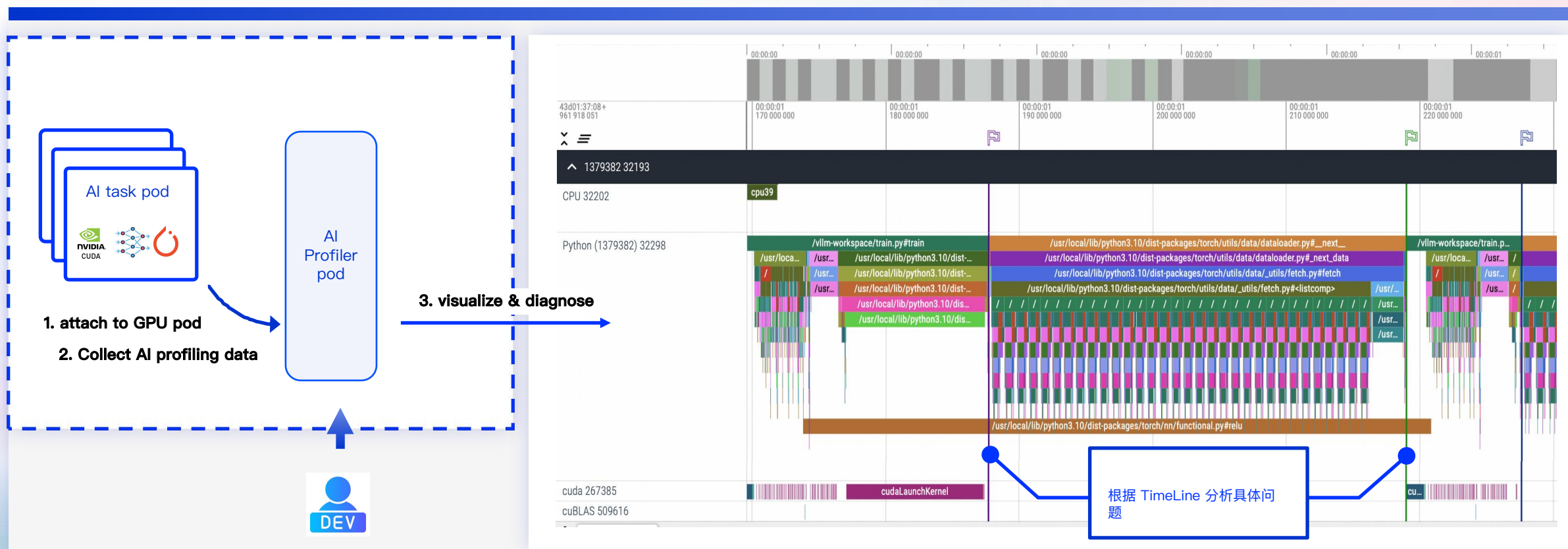
AI 业务完全无感知，Profiling 过程动态启停。使用进程动态注入与eBPF做到业务代码无侵入，且不区分业务类型。

采集丰富、TimeLine可视化聚合

采集CPU、Syscall、Python Function Call、CUDA Kernel (包含NCCL Kernel)、Torch Op等的Profiling数据，并统一到TimeLine聚合展示。

场景广、轻量级

适用与Kubernetes上通用 Containerd 场景和 Kata-Container 场景，开启AI Profiling后AI业务Overhead小于10%，。



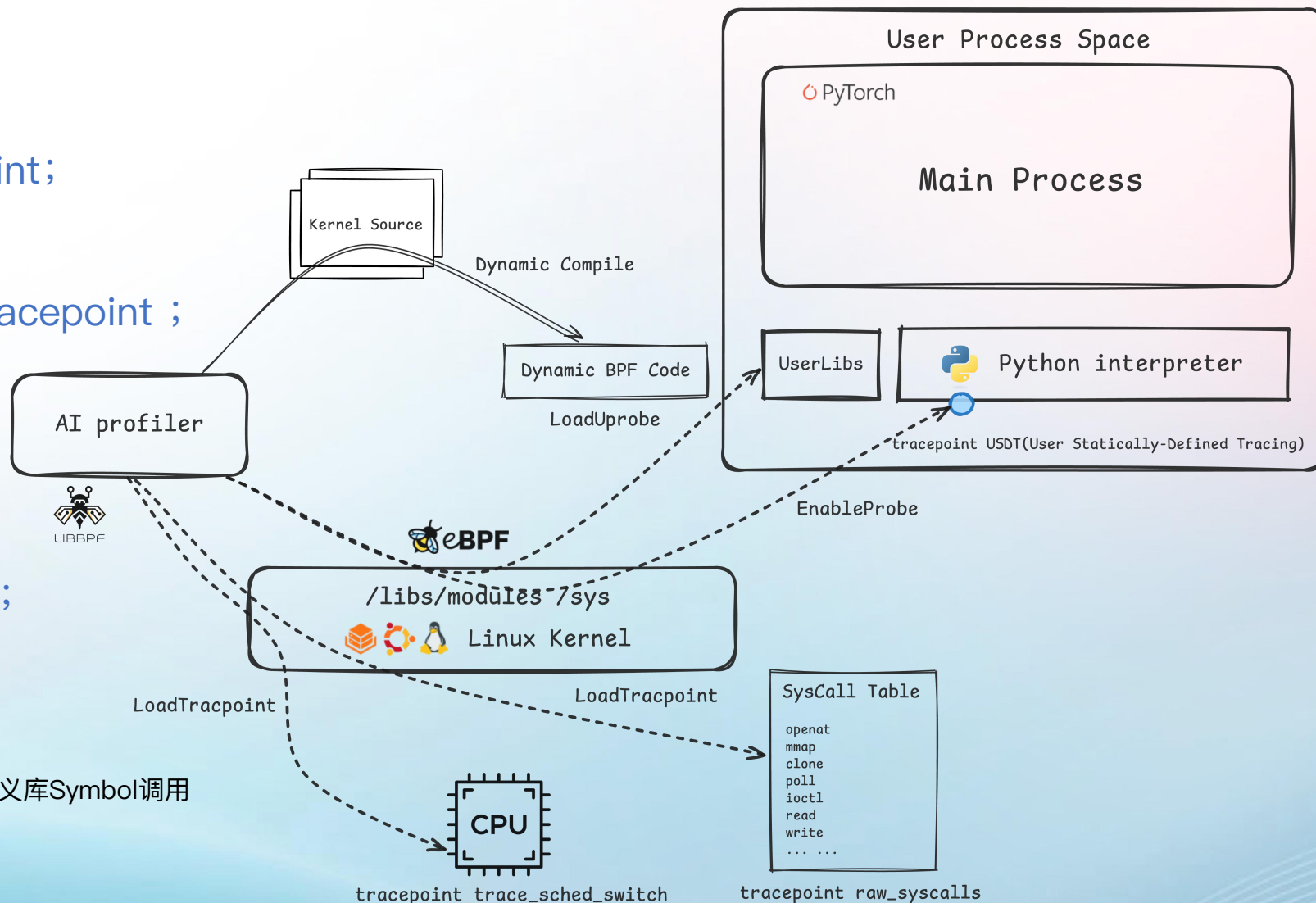
使用eBPF抓取进程通用监测数据

eBPF对目标业务进程采集项:

- ✓ CPU原生sched_switch Tracepoint;
- ✓ OS系统调用原生raw_syscalls Tracepoint ;
- ✓ 用户库Symbol Uprobe;
- ✓ Python解释器USDT Tracepoint ;

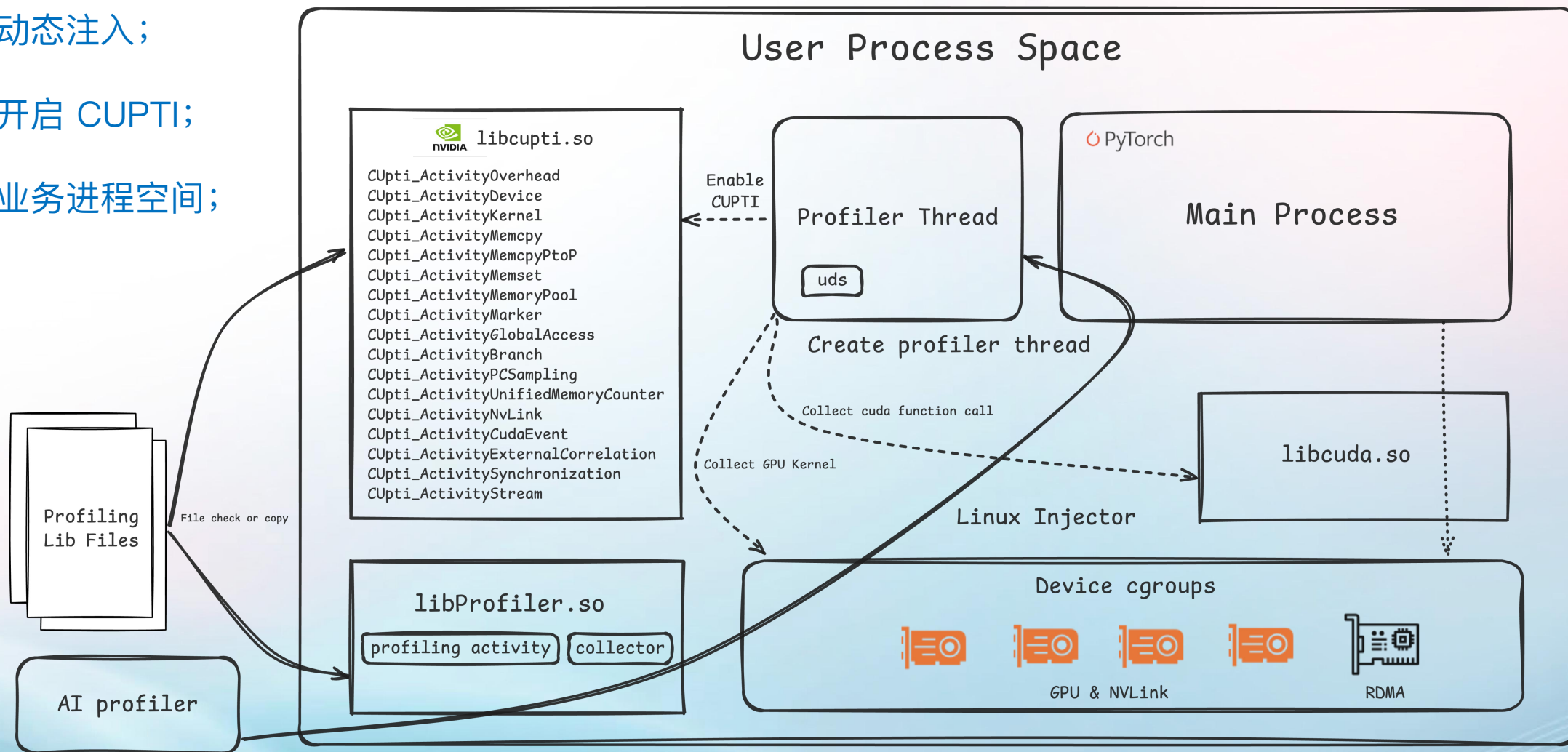
进程CPU占用、SysCall调用、Python函数调用、自定义库Symbol调用

以Chrome Tracing Events格式对采集数据进行输出



使用进程动态注入抓取GPU Kernel

- ✓ 基于动态注入；
- ✓ 动态开启 CUPTI；
- ✓ 共享业务进程空间；



面向Containerd runc设计的Profiling架构

kubernetes

Kubernetes Node

AI Application Pod

Container 0



Container 1



AI Profiling Pod

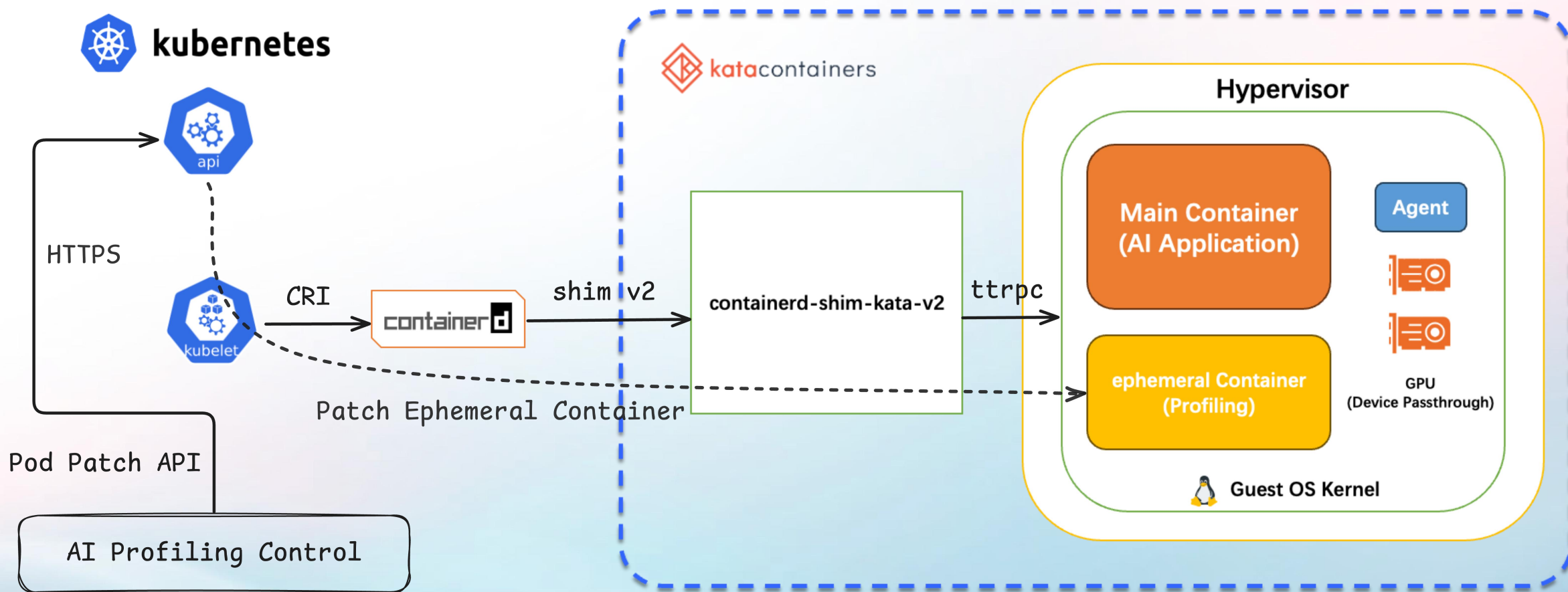
Profiling Container



OS Kernel



面向Kata-Container设计的Profiling架构



Pod Security 和 Permission Convergence



Dockerfile

```
FROM base:image

... ..

RUN apt-get update && \
    apt-get install -y sudo && \
    useradd -m -s /bin/bash profiler && \
    echo "profiler:profiler" | chpasswd

RUN mkdir -p /etc/sudoers.d && \
    echo "profiler ALL=(ALL) NOPASSWD: /usr/local/bin/ai-profiler" > /etc/sudoers.d/profiler && \
    chmod 0440 /etc/sudoers.d/profiler && \
    apt-get remove --purge -y bash && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* \

USER profiler

... ..

CMD [...]
```

- Dockerfile Build 阶段控制特定用户创建与敏感软件包移除;
- Profiler Pod 启动阶段容器权限收敛与限制目录挂载;
- Profiler 进程运行阶段限制外部Exec与可执行命令。



Profiler Pod

```
... ..

securityContext:
  capabilities:
    add :
      - CAP_BPF
      - CAP_PERFMON
      - CAP_NET_ADMIN
      - CAP_SYS_ADMIN
    runAsUser: 0

... ..

hostPID: true

... ..

- hostPath:
    path: /run/containerd/io.containerd.runtime.v2.task
    name: rootfs
- hostPath:
    path: /sys
    name: sys
- hostPath:
    path: /lib/modules
    name: modules
- hostPath:
    path: /usr/src/kernels
    name: kernels

... ..
```

03 AI Profiling关键技术 与效果验证

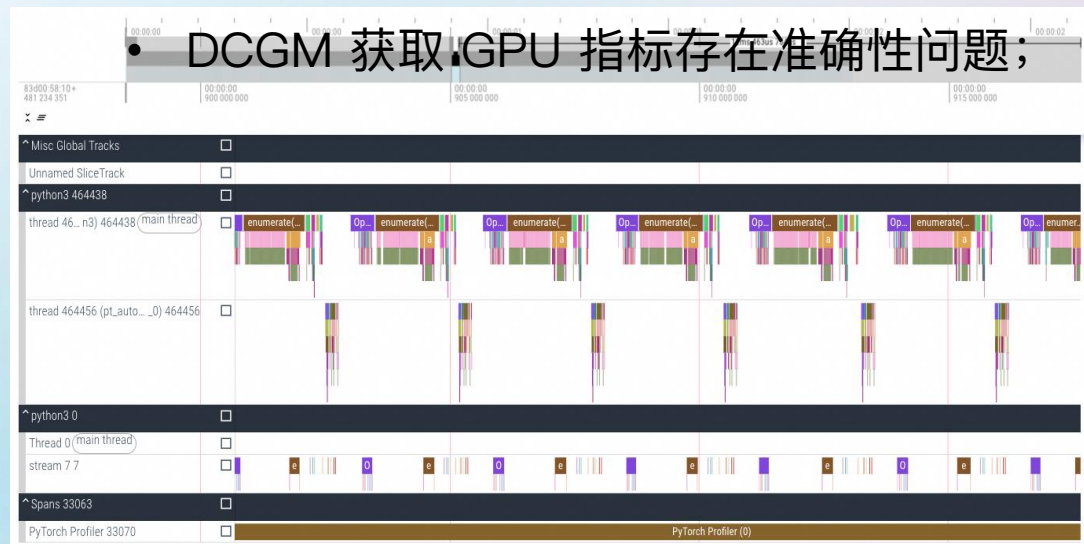
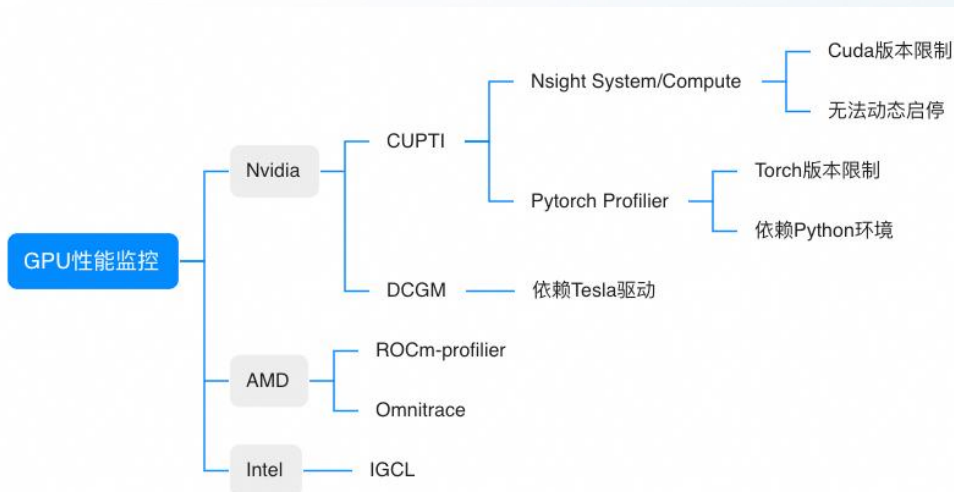
传统GPU观测手段的局限性

➤ 传统 GPU 性能采集工具的局限性：

- 硬件生态封闭，跨厂商兼容性缺失；
- 侵入性强，缺乏运行时灵活性；
- 资源开销不可控，稳定性风险高；
- 依赖链复杂，版本敏感性强；
- 自身性能瓶颈明显；

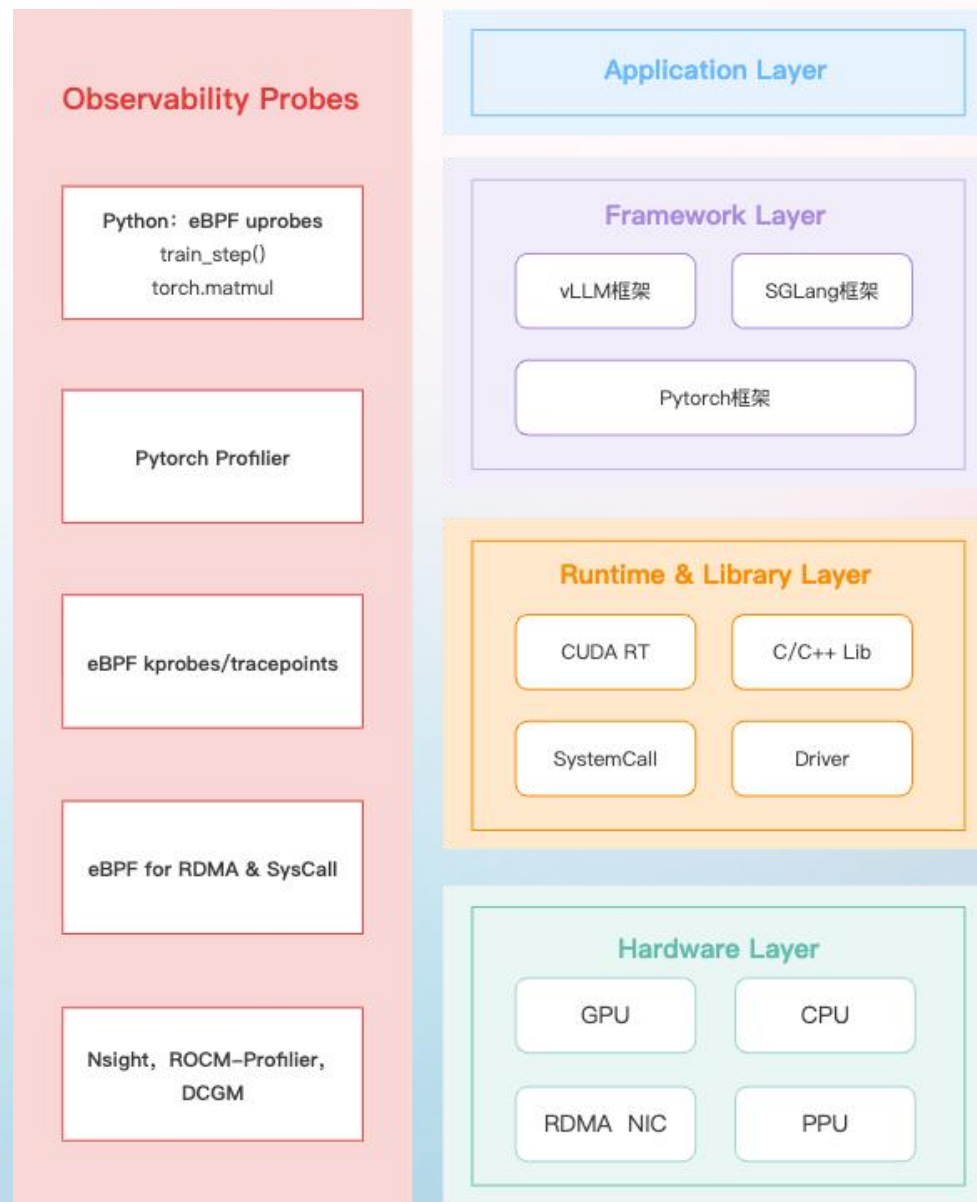
➤ 传统 GPU 性能分析的局限性：

- 观测视图割裂，关键上下文信息缺失；
- 数据规模庞大且维度复杂，缺乏高效的分析与洞察手段；
- 基于时间序列的采集模式缺乏有效对比基准；



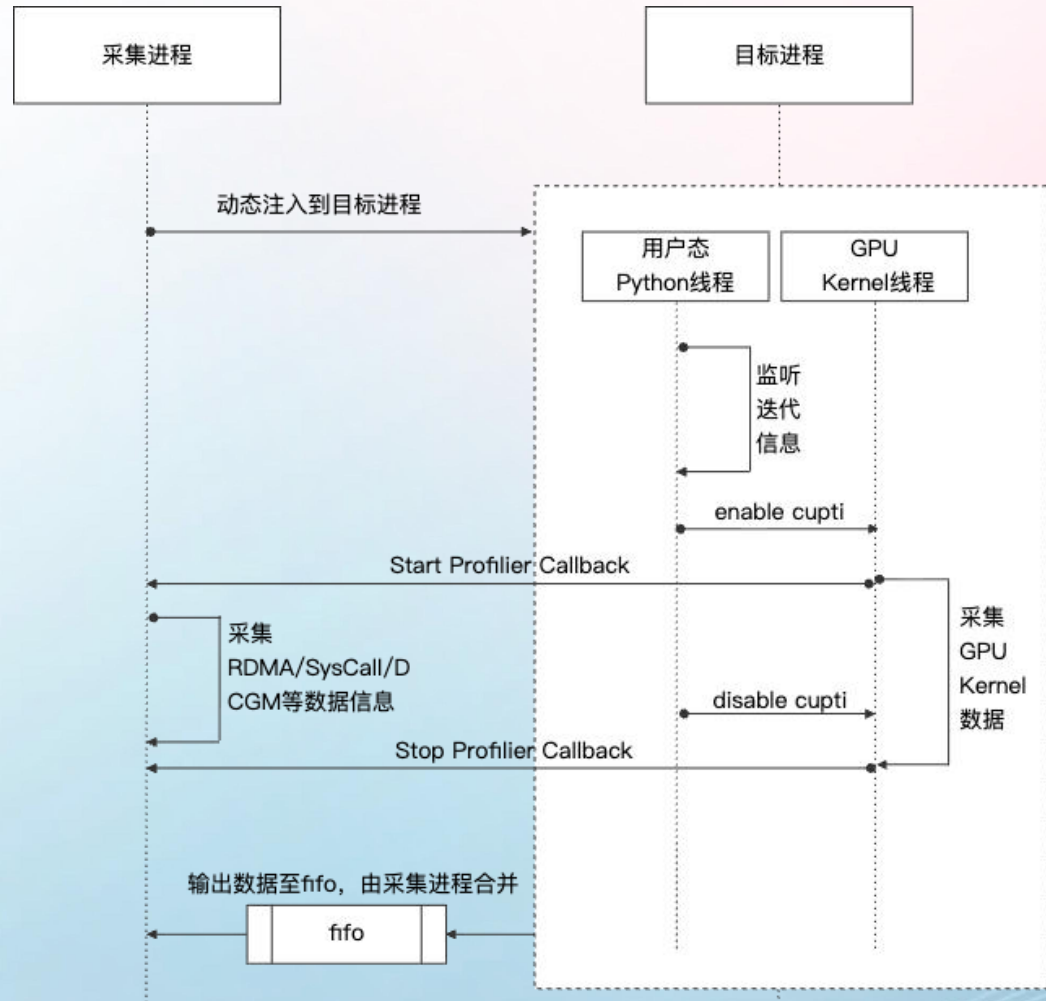
全链路采集框架

- Python 栈信息：
 - Python USDT
 - Cpython
- Torch栈信息：Pytorch Profiler
- RDMA&系统调用：
 - eBPF
- GPU Kernel 信息：
 - 基于动态注入无侵入采集 GPU Profiling 数据；
 - 统一 GPU Kernel 数据接口与回调格式，兼容不同厂商；
- GPU 监控信息：
 - DCGM, 需适配 GeForce 驱动

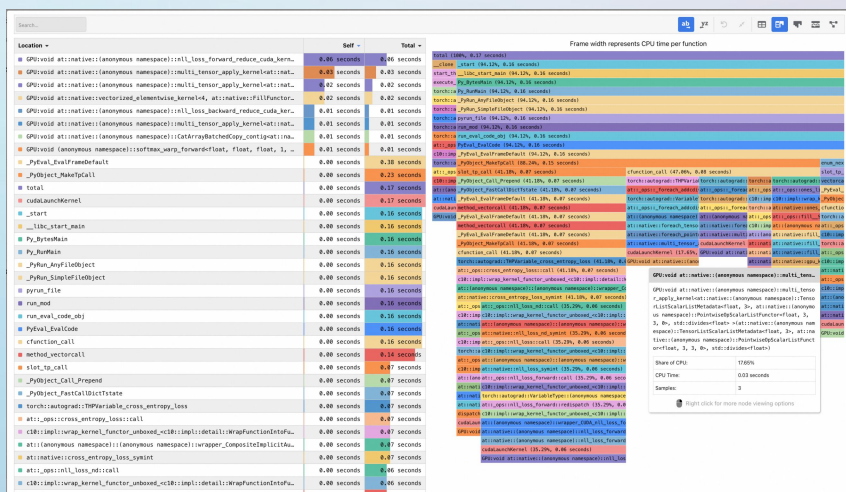
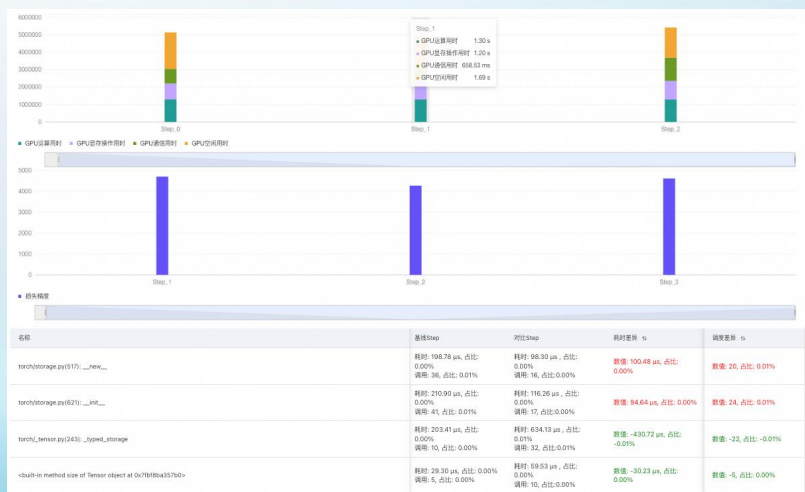
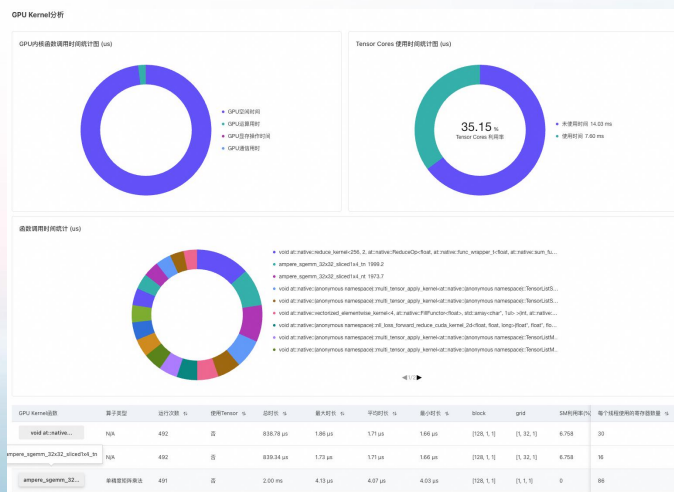


GPU Kernel信息采集流程

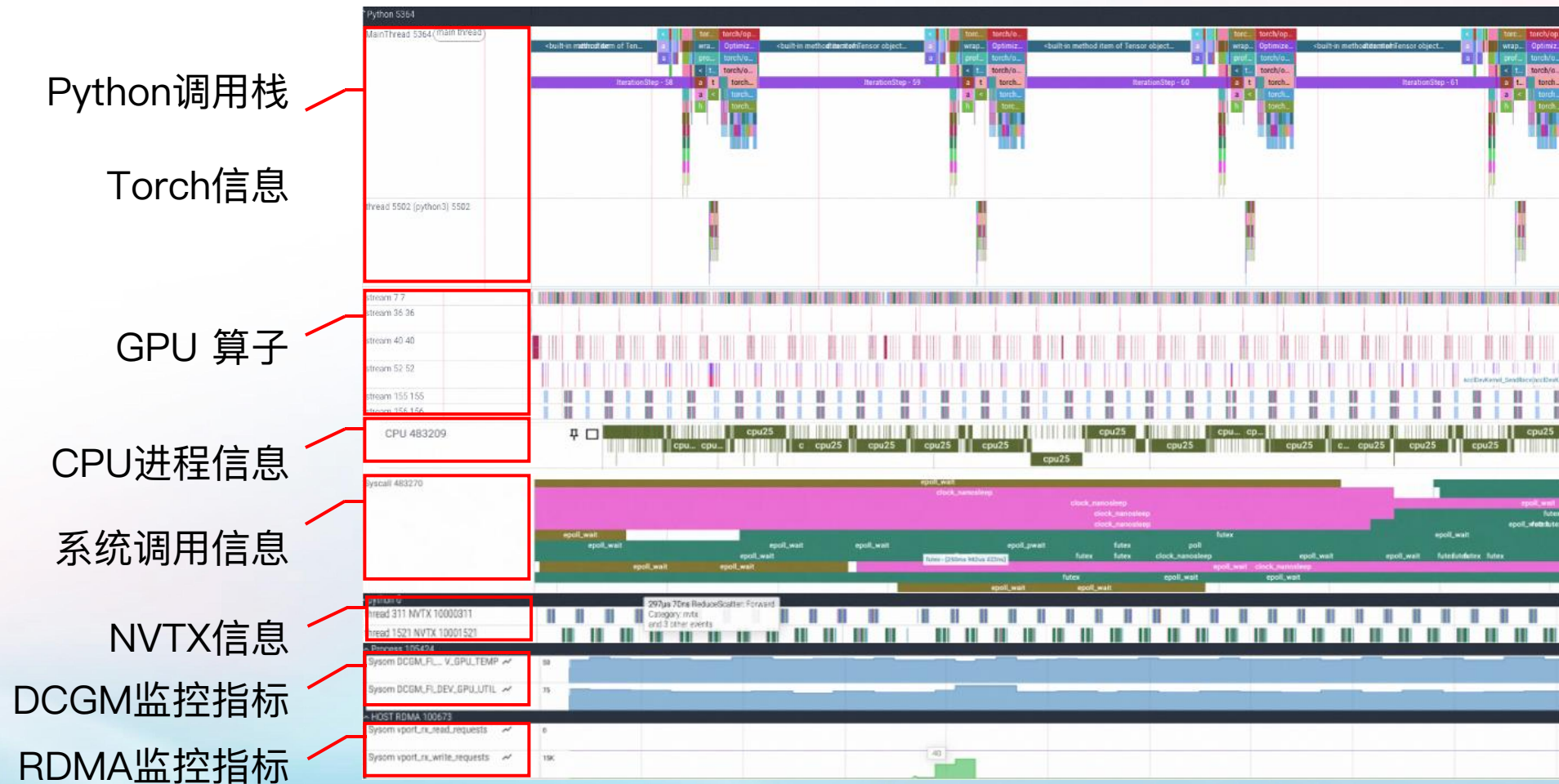
1. 动态注入目标进程，并基于预定义的语义锚点自动识别 AI 作业的迭代边界：
 - 训练任务：以 `Optimizer.step()` 调用作为单次训练迭代的结束标志；
 - 推理任务（如 vLLM）：以 `LLMEngine.step()` 作为推理迭代单元；
2. 基于迭代语义和基于时间窗口两种采集模式动态启停底层 GPU Kernel 性能采集器，确保仅在有效计算区间内采集数据；
3. 利用 GPU Kernel 启动/结束的回调事件作为全局时间锚点，同步触发其他观测模块的启停，实现跨栈指标在时间维度上的精确对齐；
4. 将来自 GPU、网络、系统及应用层的多源异构 Profiling 数据进行关联融合，上传至中心分析平台；



- 基于迭代粒度的 GPU Kernel 多维统计（计算、显存操作、通信、空闲）与差分分析（量化分析 GPU 算子在调用频次与单次/累计执行时间上的变化）；
- 对 Profiling 数据进行深度聚合，统计关键硬件资源的使用情况，包括 Tensor Core 有效计算时间占比、各类 CUDA 运行时函数及 Kernel 函数调用信息；
- 还原 CPU → CUDA → GPU Kernel 端到端调用栈，以火焰图实现性能热点可视化；



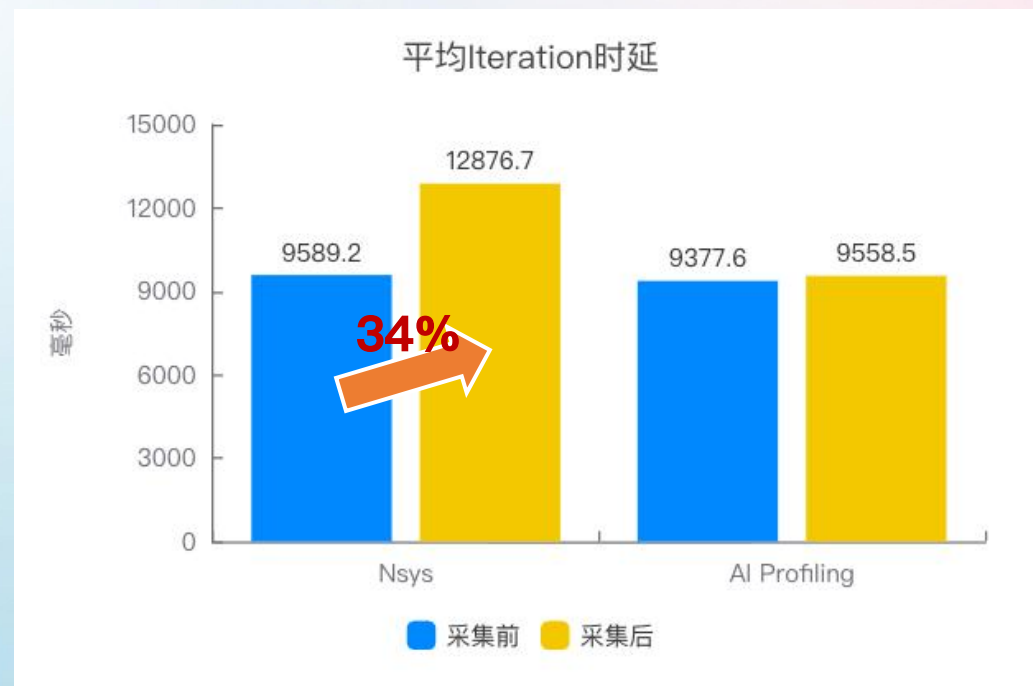
AI Profiling TimeLine



Nsys与 AI Profiling对比

当前业界普遍采用 NVIDIA Nsight Systems 作为观测和分析 AI 作业性能的主要工具。因此以下将对 Nsys 与 AI Profiling 能力进行对比分析：

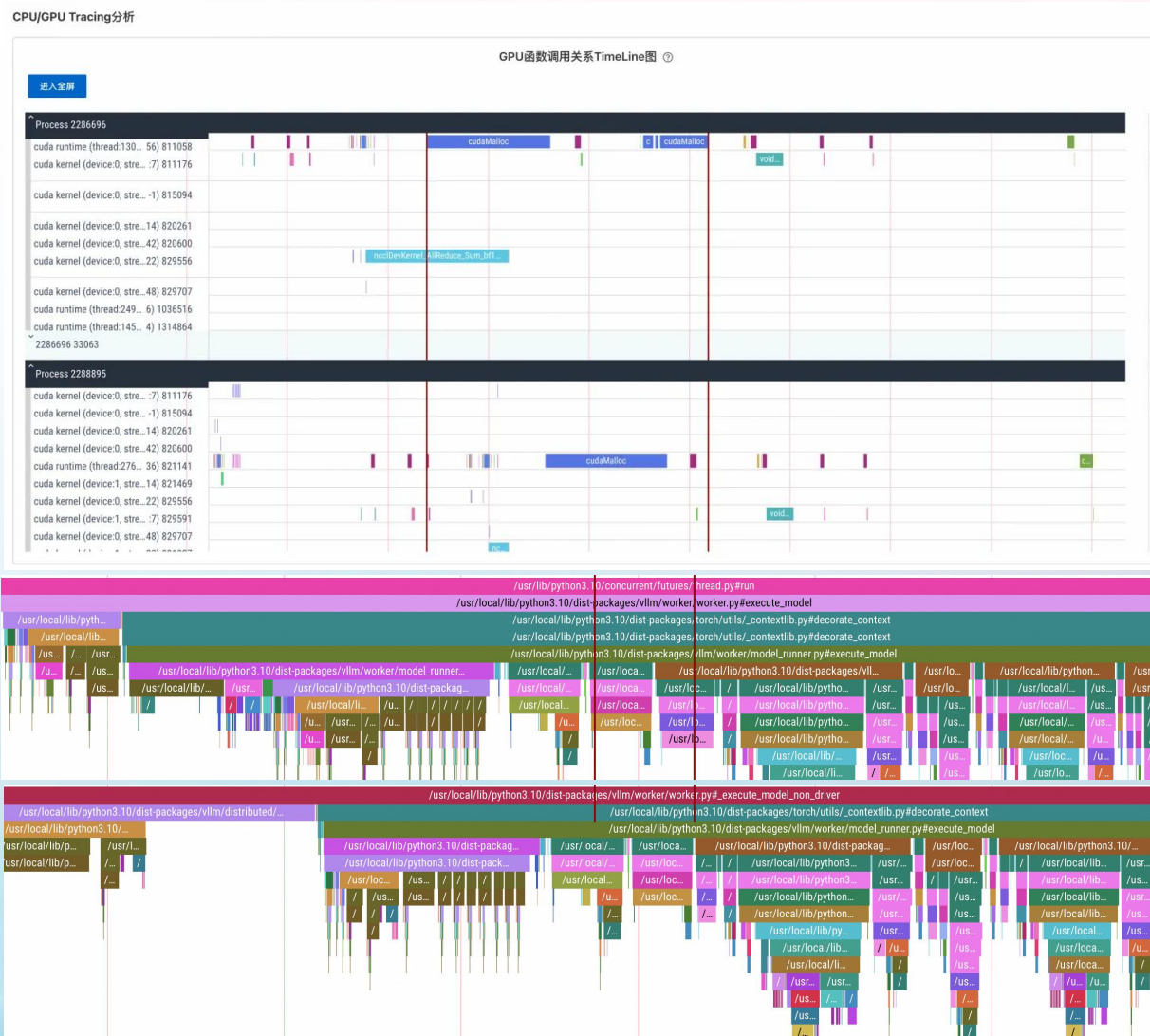
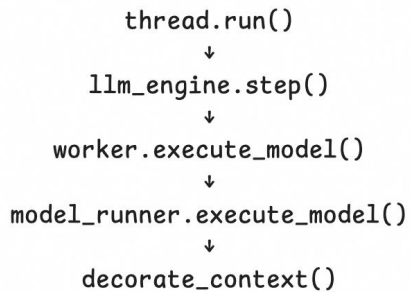
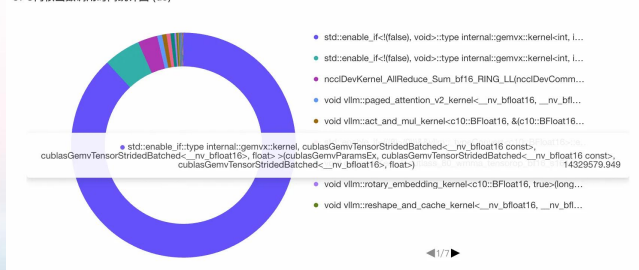
- 性能对比：Nsys 采集前后性能下降 34%， AI Profiling 采集前后性能几乎不变；
- 支持环境对比：跨 GPU 厂商兼容，支持 Python 和 C++ 进程采集，支持非 GPU 进程采集；
- 采集数据更丰富度：RDMA、SysCall、DCGM、NVTX 等；



04 案例分析

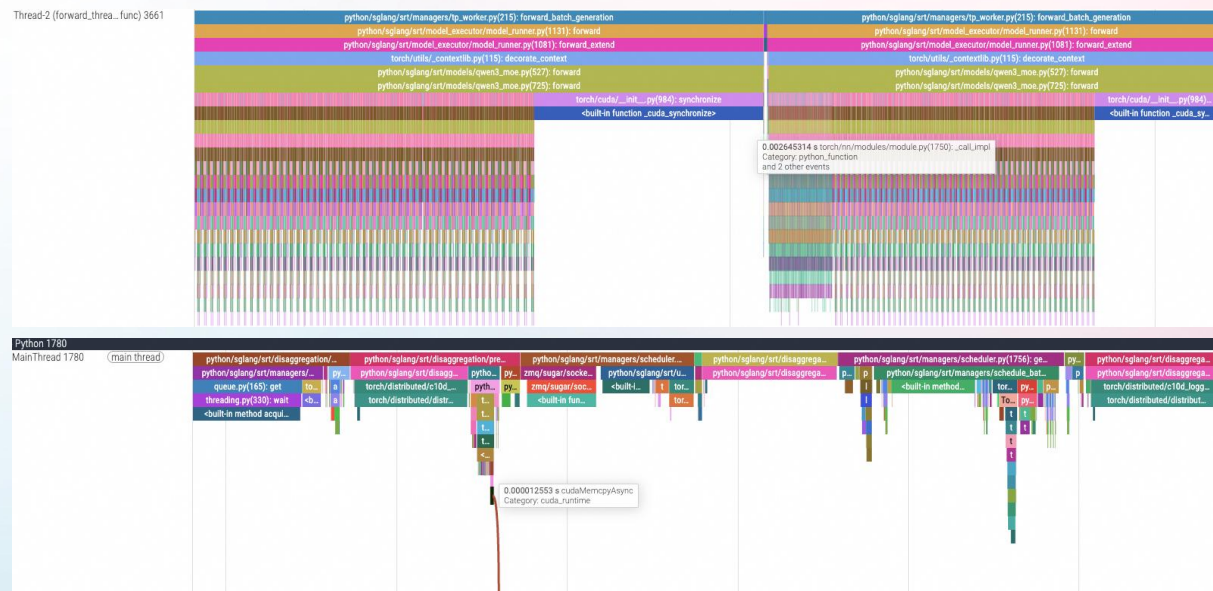


- ✓ 对在线业务应用进行AI Profiling的触发，在可视化TimeLine (Chrome Tracing)上找到显存申请的cudaMalloc 调用，打上标记线，即可找到具体的Python调用。
- ✓ 进一步定位到导致显存额外申请的调用栈如下所示，结合Python调用所指示的Torch源码进行分析，可以判断出显存增长的原因Torch的缓存管理机制，可以通过调整vLLM显存预占或Torch缓存的显存占用环境变量来进行相应的问题规避。



使用AI Profiling 观测SGlang推理过程

- 从 Python 层的 profiling timeline 可清晰识别推理请求处理的主要阶段：输入预处理 -> 模型前向计算 -> 结果后处理与响应返回。
- 在 CUDA 与 Kernel 层面，timeline 映射了上述 Python 调用所触发的底层 GPU 活动：
 - 预处理阶段：如 cuMemcpyHtoD；
 - 每轮推理 step：如 fused_moe_kernel、GEMM、activation 等；
 - 结果回传阶段：通过 cuMemcpyDtoH 将生成的结果从显存拷贝至主机内存。
- 由于部署的是多机 MoE 推理服务，CUDA 流中还密集交织着NCCL 集合通信、RDMA 跨节点传输等分布式操作。



Thanks .

